PROJECT ARCHITECTURE AND OPERATIONS

Etihad Travel Assistant

A complete guide to the local MVP: architecture, request paths, data stores, technology choices, source files, quality checks and a practical roadmap.

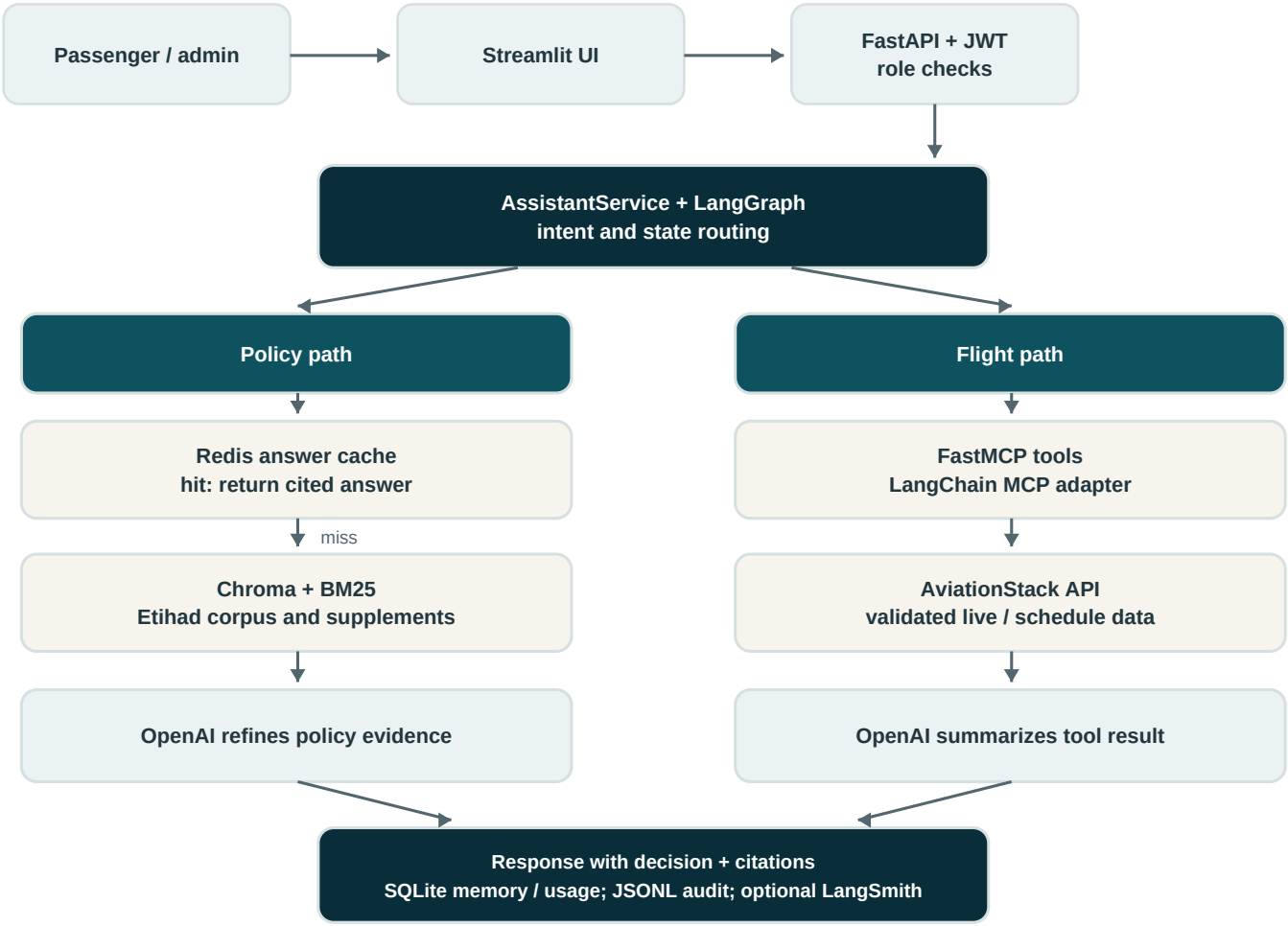
At a glance
Policy facts come from local RAG; flight facts come from AviationStack through MCP. OpenAI refines new grounded answers. Redis shares eligible policy answers; SQLite preserves user-scoped chat history and usage.

Project snapshot	Current implementation
Application	FastAPI backend and Streamlit user/admin interface
Policy corpus	55 Etihad Hub documents, 194 chunks, plus 7 official supplement chunks
Flight data	AviationStack only; no synthetic flight fallback
Model	Configurable OpenAI chat model; default gpt-4o-mini; no fine-tuning
Verification	47 tests passed; Ruff and dependency checks passed on 10 October 2026

Code review date: 10 October 2026 | Scope: local development MVP

1. Architecture at a glance

The web interface never calls a flight provider or OpenAI directly. It sends an authenticated chat request to FastAPI. The service loads conversation context, runs the LangGraph workflow, then persists the response and operational metadata.



Three different stores

Redis caches reusable policy answers. Chroma stores policy vectors, not chat memory. SQLite stores users, conversation state, messages, feedback and LLM usage.

2. Technology stack

Technology	Why it is here
Python 3.11+ / Pydantic	Implementation language and validated API models.
FastAPI / Uvicorn	HTTP backend, authentication dependencies and local server.
Streamlit / CSS	Chat UI, citations, feedback, login and admin dashboard.
LangGraph	Conditional paths for policy, flight, clarification and refusal.
LangChain / OpenAI	Prompts, chat-model calls, embeddings and MCP adapter.
Chroma + BM25	Persistent semantic policy search fused with exact-term ranking.
Redis Cloud	Shared answer cache with expiration; no SQLite cache fallback.
SQLite	Local account, conversation, feedback and usage persistence.
FastMCP / AviationStack	Live flight-status, schedule, arrival and route tools.
LangSmith / JSONL	Optional hosted traces and always-on local audit records.

Why the components are separated

Policy content, live operations data and user-specific conversation state have different freshness and privacy needs. Keeping those paths separate prevents a static policy excerpt from being mistaken for a current flight status, and prevents one user's chat history from entering the shared policy index.

- Source changes update the corpus digest and invalidate old policy-answer cache keys.
- Flight provider failures are explicit; they do not fall back to synthetic records.
- Optional tracing can be disabled without losing the local audit or usage records.

3. Request lifecycle and model calls

Request	Execution path	Chat-model call?
Repeated generic policy question	Redis returns saved answer and source IDs.	No
New policy question with evidence	Hybrid retrieval selects passages; OpenAI refines; eligible answer is cached.	Yes
Policy question without evidence	Honest refusal; no unsupported policy is invented.	No, except permitted general advice
Valid flight request	MCP invokes AviationStack; OpenAI summarizes validated data.	Yes
Missing flight detail	Clarify and persist pending intent/slot for the next turn.	No
Off-topic or unavailable provider	Decline or report no reliable provider data.	No

Embedding calls are separate from chat-model calls. On a policy cache miss, Chroma may embed the question and, on first use or corpus change, missing policy chunks. A Redis hit skips retrieval and both kinds of call.

4. How policy RAG works

The scraper converts the five Etihad Hub policy groups into titled, versioned chunks with source metadata. The repository adds separately identified official-source supplements. A BM25 lexical retriever ranks chunks; Chroma adds semantic matches and fuses both rankings. The graph then narrows the result to question-matching passages before requesting a concise, source-grounded OpenAI answer.

- A cache key includes the normalized question, corpus digest, chat model and prompt version.
- Account-specific questions and context-dependent follow-ups are not shared through Redis.
- Source IDs are stored with cached answers so citations can be reconstructed.
- If evidence is inadequate, the assistant refuses rather than guessing a policy.

Targeted source handling currently improves Economy baggage, Platinum benefits and onboard-meal questions. This is retrieval-augmented generation, not fine-tuning: the model weights are unchanged.

5. How flight questions work

The graph parses flight intent and available slots. Missing flight number, date or airport triggers a clarification; the pending intent is saved in SQLite. Once complete, the graph invokes an in-process FastMCP tool through LangChain's MCP adapter. The AviationStack adapter validates inputs, calls the provider, normalizes results, and returns a structured success, not-found, clarify or unavailable status.

Successful tool data is summarized by OpenAI, while key facts such as recognized flight status or earliest flight details are inserted from the validated tool result. Future confirmed schedules are labeled as schedules, not live operational status. A failed or empty provider result never becomes a synthetic flight.

6. Memory, cache and observability

Component	Stored information and boundary
SQLite	Argon2-hashed accounts; user-owned conversations; recent messages; pending flight state; feedback; normalized LLM usage.
Redis	Eligible policy answer draft plus cited source IDs, default seven-day TTL. Remote plaintext connections are rejected by default.
Chroma	Persistent policy embeddings and metadata. It is not the conversation-memory store.
Local JSONL	Turn decisions, IDs, retrieval metadata, latency and feedback events; not a raw chat transcript.
LangSmith	Optional graph and model traces when configured; enabling it does not itself add an LLM call.

The default context window loads up to eight recent messages from the signed-in user's conversation. Streamlit's New conversation control starts a new conversation ID; it does not erase prior SQLite rows. The current UI has no conversation-history browser.

7. Authentication, guardrails and API

FastAPI uses bearer JWTs and checks roles against the stored user record. Both user and admin roles can chat and provide feedback; audit, call records and the dashboard require admin. Guardrails decline clear off-topic requests, refuse unsupported policy claims and block individualized refund, waiver or compensation promises. These checks are rule-based and require continued evaluation.

Endpoint	Role	Purpose
POST /auth/token	Public	Sign in and obtain bearer token.
GET /me	User / admin	Read current identity.
POST /chat	User / admin	Run a policy or flight conversation turn.
POST /feedback	User / admin	Record helpful / needs-work feedback.
GET /admin/dashboard	Admin	Aggregated usage, cache and application metrics.
GET /admin/llm-calls	Admin	Recent normalized provider-call records.
GET /admin/audit-logs	Admin	Recent local JSONL audit records.
GET /health	Public	Configuration and application health summary.

8. File-by-file reference

Paths are relative to the project root. Source modules are listed individually; generated dependency and export files are grouped because they are not edited as application source.

Backend package: src/etihad_assistant

File	Responsibility
__init__.py	Package marker and version.
config.py	Settings from environment/.env; model, key, storage, Redis TLS and provider options.
schemas.py	Pydantic role, decision, citation, chat, feedback and flight contracts.
api.py	FastAPI lifecycle, CORS, login, chat, feedback, health and admin endpoints.
auth.py	JWT creation/validation and role-based dependencies.
db.py	SQLite tables and queries for accounts, conversations, messages, feedback and usage.
service.py	Constructs dependencies, runs turns, saves state and citations, writes audit events.
graph.py	LangGraph intent parsing, cache, RAG, flight, clarification and refusal paths.
guardrails.py	Travel-scope, unavailable-information and benefit-promise rules.
retrieval.py	Corpus loader, PolicyChunk model, citation conversion and BM25 retrieval.
chroma_retrieval.py	Persistent vector index, embeddings and rank fusion with BM25.
cache.py	Redis-only policy-answer cache, TLS guard, retries, TTL and stats.
ingestion.py	Etihad Hub parser and versioned corpus builder.
flights.py	AviationStack HTTP client and normalized status/schedule/arrival/route data.
flight_mcp.py	FastMCP tool definitions and in-process LangChain MCP adapter.
mcp_server.py	Optional standalone MCP server entrypoint.
llm.py	ChatOpenAI invocation and SQLite model-usage logging.
audit.py	Append-only JSONL audit writer and tail reader.

8. File-by-file reference (continued)

Interface and root

File	Responsibility
ui/streamlit_app.py	Login, chat, citations, feedback, sidebar and admin dashboard.
ui/theme.css	Etihad-inspired visual treatment and button/sidebar styles.
.streamlit/config.toml	Base Streamlit theme and toolbar options.
OIP.webp	Brand image and browser tab icon.
pyproject.toml	Package metadata, dependencies, scripts and test/lint configuration.
README.md	Local setup, flow and endpoint documentation; one setup step is stale.
.gitignore	Excludes .env, virtual environment, databases, Chroma and audit logs.
txt.example	Older sample settings, partly outdated.
n.txt	Ancillary notes containing a credential-like key; staged in Git. Do not distribute.
outputs/BUILD_REPORT.md	Historical development report; several cache descriptions are outdated.

Policy data and tests

File	Responsibility
data/corpus/etihad_hub_policy_library.json	6 October 2026 Etihad Hub snapshot: 55 records, 194 chunks.
data/corpus/etihad_official_economy_baggage.json	One official Economy baggage supplement chunk.
data/corpus/etihad_official_guest_platinum.json	Three official Platinum-benefit supplement chunks.
data/corpus/etihad_official_onboard_meals.json	Three official onboard/special-meal supplement chunks.
tests/test_api.py	Authentication, roles and chat endpoint tests.
tests/test_core.py	Routing, RAG, cache, guardrail, follow-up and usage regressions.
tests/test_aviationstack.py	Provider and MCP normalization, error and no-synthetic-data tests.
tests/test_storage.py	Redis, Chroma and user-scoped history tests.

Generated or local working files

data/app.db is the local SQLite store; data/chroma/ is the persisted vector index; logs/audit.jsonl is the audit stream. work/policy-library.html is a saved source-page capture; the other work/*.db and work/*.jsonl files are local working artifacts. The seven outputs/*.zip files are historical export snapshots, not runtime dependencies. The .venv/, .pytest_cache/, .ruff_cache/ and .git/ directories are tooling state.

9. Running and verifying locally

The app uses two terminals. Start FastAPI with: `.\.venv\Scripts\Activate.ps1`, then `uvicorn etihad_assistant.api:app --reload --host 127.0.0.1 --port 8000`. Start Streamlit in another terminal with: `streamlit run ui/streamlit_app.py`. Open http://localhost:8501; inspect http://127.0.0.1:8000/health for backend status.

Required or optional .env variable names include OPENAI_API_KEY, AVIATIONSTACK_API_KEY, REDIS_URL, REDIS_REQUIRE_TLS, RAG_BACKEND, LANGSMITH_TRACING, LANGSMITH_API_KEY and JWT_SECRET. Do not copy secret values into source files or exported ZIPs.

Documentation mismatch
README.md asks new users to copy .env.example, but that file is absent from the current working tree. Use the existing ignored .env for this installation and restore a safe example file before distributing the project.

10. Current boundaries

- No Azure Foundry, model fine-tuning, booking transactions or account-linked airline actions.
- Policy sources are snapshots; publication may have changed since their checked dates.
- The intent detector and promise guardrail are rules, not comprehensive classifiers.
- The UI does not provide a conversation-history browser even though SQLite stores history.
- The admin dashboard tracks usage, not authoritative billing cost or provider uptime.
- Tests use controlled fakes and do not confirm live external-account access.

11. Prioritized improvement plan

Priority	Recommendation and reason
P0 - security	Rotate the credential-looking key in staged n.txt; remove it from Git/index and shared archives. Replace demo secrets; add a real password-update flow and rate limiting.
P1 - correctness	Build representative evaluations for intent, MCP arguments, retrieval relevance, citation grounding, refusals and multi-turn follow-ups.
P1 - freshness	Refresh policy sources with review, record effective dates, detect changes and expose source age in answers.
P1 - flight UX	Handle airport/time-zone ambiguity more broadly and render validated provider data deterministically if OpenAI summarization fails.
P2 - RAG	Check individual high-stakes claims against cited passages; tune retrieval thresholds; version cache against retrieval/index changes.
P2 - interface	Add conversation listing/resume, useful feedback reasons and admin quality metrics.
Later	Only if moving beyond local MVP: managed secrets, migrations/backups, retention, monitoring and deployment hardening.

12. Key takeaways

In one sentence
This is a policy-grounded, flight-data-aware local assistant: RAG supplies policy evidence, AviationStack supplies flight facts, OpenAI refines first-time answers, Redis reuses eligible answers, and SQLite remembers each user's conversation.

The highest-priority action is security cleanup of the staged credential-like text. The best quality investment after that is a repeatable evaluation dataset built from real questions and failures; fine-tuning is not a prerequisite. A source-grounded answer can still be wrong if retrieval chooses the wrong evidence or a source is stale.

Source and review notes

This document summarizes the files in the local project as inspected on 10 October 2026. Source code is the authority where README.md or BUILD_REPORT.md describe older behavior. No live flight status, Redis Cloud connection or current OpenAI response was tested for this document.

Official Etihad policy sources recorded by the project:
https://www.etihadhub.com/en-ae/policy-library
https://www.etihad.com/en-in/help/faq/fares
https://www.etihad.com/en/etihadguest/tiers-and-benefits
https://www.etihad.com/en/plan/fly-with-etihad/onboard-services/meals
https://www.etihad.com/en/help/special-assistance/special-meals

Further reading for the roadmap:
https://developers.openai.com/api/docs/guides/evaluation-best-practices
https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html

Verification: 47 pytest tests passed, Ruff passed, and pip check found no broken requirements. One LangChain MCP beta warning remains.